4-bit Array Multiplier

From RTL to GDSII with Open-Source Tools

Veekshita Sai 2024102026
Chandini Gayathri 2024102020

Abstract

This report presents the design and implementation of a 4-bit unsigned array multiplier using Verilog and open-source tools for the SkyWater SKY130 process. The design was verified for all 256 input combinations and synthesized into 73 standard cells. Static timing analysis showed a minimum clock period of 2.596 ns. After place and route, the design achieved a 7.0 ns clock period with positive slack, zero DRC errors, and zero LVS errors.

1 Introduction

Multipliers are important digital circuits used in processors, DSP systems, and machine-learning hardware. This project implements a 4×4 unsigned array multiplier that produces an 8-bit product using partial products and full adders.

The design follows the RTL-to-GDSII flow, which converts Verilog code into a physical chip layout. Using the SKY130 PDK and open-source tools such as Yosys, OpenSTA, OpenROAD, Magic, KLayout, Netgen, and LibreLane, we perform synthesis, placement, routing, timing analysis, and layout verification to evaluate the design's area, speed, and correctness.

2 Objectives

1. Design and verify a 4-bit array multiplier in Verilog using full adders, registered inputs, and a registered output.
2. Synthesize the design using Yosys and verify its functionality through gate-level simulation.
3. Analyze the critical path, minimum clock period, and maximum frequency using OpenSTA.
4. Complete the RTL-to-GDSII flow using LibreLane and obtain a DRC-clean and LVS-clean layout.
5. Compare area and timing results across different stages of the design flow.

3 Design Overview: how a 4-bit array multiplier works

3.1 Partial Products

A 4-bit array multiplier generates four partial products by ANDing each bit of B with all bits of A and shifting the results according to the bit position. Adding these partial products produces the 8-bit product.

3.2 The Array Structure

The multiplier adds four partial products row by row using full adders. Each row produces a product bit, while the remaining sum bits and carry are passed to the next row. The final row produces the upper four bits of the 8-bit product. The first row requires no full adders.

3.3 Size and Delay

A 4-bit array multiplier uses 16 AND gates and 12 full adders. Its area grows approximately as N^2, while its delay grows approximately linearly with the number of bits N.

3.4 Registers, Latency and Throughput

The design has registered inputs and a registered output. The input is captured at one clock edge, and the product is captured at the next edge, giving a latency of 2 clock cycles. A new input pair can be accepted every cycle, providing a throughput of one result per clock cycle.

4 Design Specification

The `mult_array` module has the following ports:

The design uses 16 flip-flops, 16 AND gates, and 12 full adders. It has a latency of 2 clock cycles and a throughput of one result per cycle. The RTL uses full-adder instances instead of the multiplication operator.

Table 1: Module ports.

Port	Direction	Width	Description
`clk`	Input	1	Clock
`rst_n`	Input	1	Active-low reset
`a`	Input	4	Multiplicand
`b`	Input	4	Multiplier
`p`	Output	8	Registered product

Table 2: Tools used in the design flow.

Tool	Purpose
Icarus Verilog	RTL and gate simulation
GTKWave	Waveform viewing
Yosys	Logic synthesis
OpenSTA	Timing analysis
LibreLane / OpenROAD	Physical design and routing
Magic / Netgen	Layout verification
KLayout	GDSII viewing

5 Technology and Tools

The design uses the SkyWater SKY130 130 nm process and the `sky130_fd_sc_hd` standard-cell library. Timing analysis uses the typical corner `tt_025C_1v80`, while physical design signoff checks slow and fast corners, including `ss_100C_1v60` and `ff_n40C_1v95`.

All tools were run on Ubuntu using the IIC-OSIC-TOOLS Docker environment. The SKY130 PDK was selected using `sak-pdk sky130A`.

6 Methodology

The design was completed in four stages:

1. **RTL design and verification:** Designed the multiplier in Verilog and tested all 256 input combinations using Icarus Verilog.

2. **Logic synthesis:** Used Yosys to generate the SKY130 gate-level netlist and report cell area.

3. **Static timing analysis:** Used OpenSTA to determine the critical path, minimum clock period, and maximum frequency.

4. **Physical design:** Used LibreLane to perform floorplanning, placement, clock-tree synthesis, routing, and signoff.

The same self-checking testbench was used for RTL and gate-level simulation to verify that synthesis preserved the design's functionality.

7 Part A: RTL design and verification

7.1 Overview

The 4-bit multiplier was implemented in Verilog and tested for all $16 \times 16 = 256$ possible input combinations. This checks the functional behaviour before synthesis and physical design.

7.2 Design approach

The design contains two modules. The `full_adder` module adds three one-bit inputs and produces a sum and carry. The `mult_array` module uses AND gates to form partial products and full adders to add them in rows. It contains 16 AND gates and 12 full adders. The inputs are stored in registers, and the 8-bit product is stored in an output register. An asynchronous active-low reset clears the registers. The input and output registers give the design a latency of two clock cycles. The multiplier is built from gates and adders; the multiplication operator is used only by the testbench to calculate expected results.

Figure 1 shows the complete structure of `mult_array`: the input registers, all 16 AND gates, all 12 full adders and the output register. Each row of full adders is drawn one step to the right of the row above it. In this way every sum bit goes straight down into the adder below it, and every carry goes to the right inside its row. AND gate (i, j) makes $pp_i[j] = a_r[j] \cdot b_r[i]$, and FA_{ij} is the full adder in row i, bit j. The product bit $p[k]$ comes out of column k.

7.3 The testbench

The self-checking testbench applies every input pair from 0 to 15, waits for the two-cycle latency, and compares the output with the expected product. It also records a waveform and reports the number of errors.

7.4 Results

The simulation passed all 256 input combinations with zero errors. To check that the testbench could detect a fault, the sum equation was intentionally changed to omit the carry-in term. The testbench then reported 95 errors. For example, inputs 3 and 3 produced 5 instead of 9, while inputs 3 and 7 produced 9 instead of 21. The original equation was restored, and the testbench passed again.

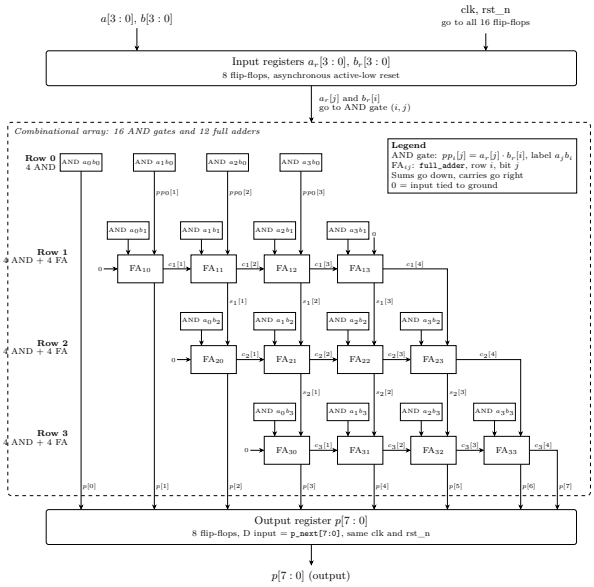


Figure 1: Block diagram of `mult_array`: input registers, 16 AND gates, 12 full adders (rows 1 to 3) and the output register. Sums go down, carries go to the right, and product bit $p[k]$ comes out of column k.

Hand-worked example. The assignment says to take A as the last digit of a team member's roll number plus 5. We used Veekshita's roll number, 2024102026. Its last digit is 6, so $A = 6 + 5 = 11$ (binary 1011). We used $B = 7$ (binary 0111). The multiplication by hand is:

```
        1 0 1 1     A  = 11
      x 0 1 1 1     B  = 7   (b0 = 1, b1 = 1, b2 =
  1, b3 = 0)
    ---------------
        1 0 1 1     pp0 = A AND b0  (b0 = 1)
      1 0 1 1       pp1 = A AND b1  (b1 = 1),
  shifted 1
    1 0 1 1         pp2 = A AND b2  (b2 = 1),
  shifted 2
  0 0 0 0           pp3 = A AND b3  (b3 = 0),
  shifted 3
    ---------------
0 1 0 0 1 1 0 1     P = 77
```

As a check, $11+22+44+0 = 77$, and $01001101_2 = 64 + 8 + 4 + 1 = 77$.

Finding it in the waveform. The testbench applies vector $k = 16a + b$ at time $20 + 20k$ ns. For $a = 11$ and $b = 7$, $k = 183$, so the inputs change at 3680 ns. The input registers capture them at 3685 ns, and the output register stores the product at 3695 ns. Thus, p becomes 77 after two rising clock edges. In the waveform, p changes from 66 to 77 at 3695 ns and then to 88 at 3715 ns.

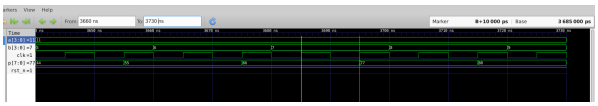


Figure 2: Waveform of the 11×7 example (`clk`, `rst_n`, `a`, `b`, and `p`, shown in decimal). The markers indicate rising clock edges at 3685 ns and 3695 ns. The product 77 appears two clock edges after the inputs, confirming the 2-cycle latency.

7.5 Observations

- All 256 input combinations passed, including $0\times$anything and $15 \times 15 = 225$.
- The waveform confirms the two-cycle latency.
- The reset clears the registers and output to zero.
- The intentional fault caused 95 errors, confirming that the testbench detects mistakes.

8 Part B: Logic synthesis

8.1 Overview

Synthesis converts the verified RTL into a netlist of SKY130 standard cells. We checked the cell count, area, and whether the synthesized netlist still produces the correct outputs.

8.2 The idea behind it

Synthesis first converts the RTL into generic logic and simplifies it. Technology mapping then replaces the generic logic with cells available in the SKY130 library. In Yosys, `dfflibmap` maps flip-flops, while `abc` maps the remaining logic.

We also used flattening to combine `mult_array` and its `full_adder` instances into one logic block. This allows the tool to simplify logic across module boundaries.

8.3 The synthesis flow

The synthesis script performs seven steps: read the RTL files, synthesize and flatten the design, map flip-flops, map the remaining logic, remove unused logic, generate the cell and area report, and write the gate-level netlist. The mapping uses the typical-corner SKY130 library `sky130_fd_sc_hd__tt_025C_1v80.lib`. We ran the script using Yosys.

8.4 Results

Table 3: Part B results.

Metric	Value
Total cells	73
Flip-flops	16 (`dfrtp_1`)
Total cell area	805.773 μm^2
Flip-flop area	400.384 μm^2
Flip-flop area (%)	49.69%
Common cell types	`dfrtp_1` × 16, `nand2_1` × 11, `xnor2_1` × 4
Simulation result	`PASS`: 256 vectors, 0 errors

The 73 cells are grouped by function below.

The logic uses 57 cells with an area of 405.389 μm^2, while the 16 flip-flops use 400.384 μm^2. Thus, the logic and registers each account for about half of the total area.

8.5 Checking the synthesized netlist

We tested the synthesized netlist using the same testbench as Part A and the SKY130 functional cell models. All 256 input combinations passed, with zero errors. This confirms that the netlist matches the expected outputs for the tested inputs, although it is not a formal equivalence proof.

8.6 Observations

- **Flip-flops are preserved.** The netlist contains 16 flip-flops, matching the four input bits of each operand and the 8-bit output register.
- **Logic is remapped.** The 16 AND gates and 12 full adders in the RTL become 57 library logic cells. The tool combines and simplifies the logic instead of mapping each RTL component to one cell.
- **Constant inputs simplify logic.** Adders with a carry-in or input tied to 0 can be simplified, reducing the required logic.
- **NAND gates are common.** The netlist contains 11 `nand2_1` cells.
- **Cell selection depends on the library.** The tool also uses three low-power-flow cells as part of the mapped logic.
- **Registers use significant area.** The flip-flops account for 49.69% of the total cell area.
- **Internal names are generic.** After cleanup, some cells have names such as `_118_`, so the netlist must be inspected to identify registers during timing analysis in Part C.

9 Part C: Static timing analysis

9.1 Overview

In this part, we used OpenSTA to find the minimum clock period (T_{min}), maximum clock frequency (F_{max}), and critical path of the synthesized netlist. We also compared the critical path with the estimate of about $2N = 8$ full-adder delays.

9.2 The idea behind it

Static timing analysis (STA) calculates delays along register-to-register paths without simulating input patterns. It uses the cell delays in the Liberty file to check whether each path meets the clock timing requirement.

The main terms are:

- **Clock-to-Q delay:** Time for a flip-flop output to settle after a clock edge.
- **Setup time:** Time for which data must be stable before the next clock edge.
- **Arrival time:** Time taken for data to reach the destination flip-flop.
- **Required time:** Latest time the data may arrive.
- **Slack:** Required time minus arrival time. Positive slack means timing is met; negative slack means timing fails.
- **WNS:** The worst slack among the paths, which identifies the critical path.

For an ideal clock, the minimum period and maximum frequency are:

$$T_{min} = T - \text{WNS}, \qquad F_{max} = \frac{1000}{T_{min}}\ \text{MHz} \quad (T_{min} \text{ in ns}).$$

We first used a 10 ns clock period to find the limit, then ran STA again at the rounded minimum period to check that the slack was close to zero.

The analysis uses the typical SKY130 corner, `tt_025C_1v80` (25°C, 1.8 V), with an ideal clock and no wire delay.

9.3 The OpenSTA flow

The OpenSTA script loads the SKY130 timing library and synthesized netlist, links the design, defines the clock and input/output delays, and reports the worst slack and critical path. The clock period is controlled by the variable `T`. We ran the script with OpenSTA and used a final period of $T = 2.60$ ns.

9.4 Analysis and results

With a relaxed clock period of $T = 10$ ns, OpenSTA reported a worst slack of 7.404 ns. Therefore,

$$T_{min} = 10 - 7.404 = 2.596 \text{ ns}, \qquad F_{max} = \frac{1000}{2.596} \approx 385.2 \text{ MHz}.$$

We then set the period to 2.60 ns, rounded to 0.01 ns. The resulting slack was +0.004 ns, which is almost zero and confirms the estimate.

The critical path starts at flip-flop `_118_`, corresponding to input register bit `b_r[0]`, and ends at flip-flop `_112_`, corresponding to output bit `p[6]`. The cell delays along this path are shown below.

The data arrives at 2.460 ns, and the required time is 9.865 ns, giving 7.404 ns slack. The asynchronous reset paths are checked using recovery timing. Their slack is 9.228 ns at $T = 10$ ns and 1.828 ns at $T = 2.60$ ns, so they are not critical.

The multiplier has a two-cycle latency, so its latency at the maximum frequency is $2 \times T_{min} = 5.192$ ns. It can produce one result per cycle, giving a throughput of 385.2 million results per second.

Table 4: Part C results.

Metric	Value
Worst slack ($T = 10$ ns)	7.404 ns
$T_{\min}$	2.596 ns
$F_{\max}$	385.2 MHz
Slack at 2.60 ns	+0.004 ns
Critical path start	`b_r[0]` (`dfrtp_1`)
Critical path end	`p[6]` (`dfrtp_1`)
Critical path cells	8 logic cells (10 including flip-flops)
Latency ($2T_{\min}$)	5.192 ns
Throughput	385.2 million results/s

The critical path contains these logic cells in order: `and2_0` → `or3_1` → `and4_1` → `o2111a_1` → `o21ai_0` → `xnor3_1` → `maj3_1` → `xor3_1`.

9.5 Comparison with the estimate of about $2N$ full-adder delays

The expected path length is about $2N = 8$ full-adder delays. Our critical path also has eight logic cells, but these are not eight complete full adders. Three cells, `xnor3_1`, `maj3_1`, and `xor3_1`, perform adder-like functions. The other five cells handle partial-product and carry logic. The three adder-like cells contribute 0.827 ns of the total 2.065 ns logic delay.

The average delay per logic cell is about $2.065/8 = 0.26$ ns. The path is shorter than a chain of eight complete ripple-carry adders because synthesis flattened the design and combined logic into complex gates. Therefore, the $2N$ estimate helps predict the path through the array, but the final delay depends on the mapped cells.

9.6 Observations

- **Clock period:** The minimum period is 2.596 ns, giving a maximum frequency of 385.2 MHz.
- **Delay breakdown:** Logic contributes 2.065 ns (about 79.5%), while register overhead contributes 0.530 ns (about 20.4%).
- **Critical path:** The path starts at `b_r[0]` and ends at `p[6]`, passing through eight logic cells.
- **Cell delays:** `maj3_1` (0.375 ns) and `or3_1` (0.361 ns) are the slowest cells on the path; `xor3_1` (0.178 ns) is the fastest.
- **Setup analysis:** This part covers setup timing only. Hold timing is checked later during LibreLane signoff.
- **Ideal conditions:** The results use the typical corner, an ideal clock, and no wire delay, so actual silicon may be slower.

10 Part D: Physical design (RTL to GDSII)

10.1 Overview

In this part, LibreLane converts the synthesized design into a physical layout by placing cells, building the clock tree, routing wires, and checking manufacturing rules. The goal is a GDSII layout with zero DRC and LVS errors and non-negative setup slack.

10.2 The idea behind it

LibreLane runs the RTL-to-GDSII flow using tools such as Yosys, OpenROAD, Magic, KLayout, and Netgen. The main stages are summarized below.

Timing depends on operating conditions. The flow checks typical, slow, and fast corners. The slow corner is usually hardest for setup timing because its gates are slower. DRC checks layout against manufacturing rules, while LVS compares the extracted layout circuit with the netlist. Antenna checks identify wires that may collect charge during manufacturing. Tap cells connect the substrate to power rails; decap and fill cells occupy empty row space; buffers help distribute the clock or repair timing.

10.3 Configuration

The configuration specifies the design name, RTL files, clock port, clock period, and fixed die size of 150×150 μm. We used absolute floorplan sizing because the design is small. The final clock period was set to 7.0 ns after testing different values. LibreLane was run from the `pnr/` folder. The final metrics and GDSII file were copied from the successful run; the large run directory was not included in the repository.

10.4 Finding a working clock period

The starting period was $1.2 \times T_{min}$ from Part C, rounded up to the next 0.5 ns. Since $T_{min} = 2.596$ ns, this gave 3.5 ns. The first seven runs failed setup timing, while the eighth run passed at 7.0 ns.

Table 5: The eight place-and-route runs.

Run	Period (ns)	Slack (ns)	Result
1	3.5	−2.144	FAIL
2	4.0	−1.644	FAIL
3	4.5	−1.144	FAIL
4	5.0	−0.644	FAIL
5	5.5	−0.144	FAIL
6	6.0	−0.082	FAIL
7	6.5	−0.165	FAIL
8	**7.0**	**+0.335**	**PASS**

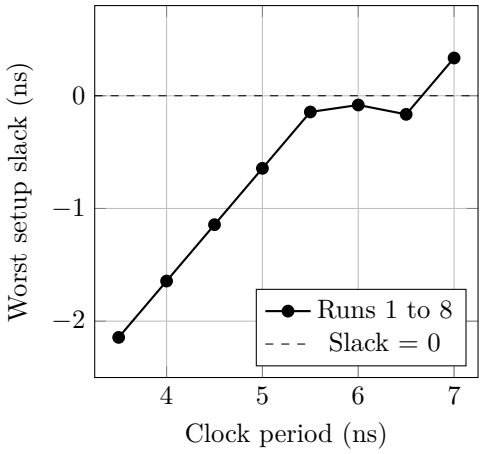


Figure 3: Worst setup slack against clock period for the eight runs. The slack improves one to one with the period at first, then varies slightly and becomes positive at 7.0 ns.

The first run failed because the typical-corner slack was positive (+0.568 ns), but the slow corner slack was −2.144 ns. Unlike Part C, physical signoff includes slow operating conditions, real wires, and the clock tree. The slack trend also changed after 5.5 ns. We did not confirm the cause; one possibility is that the flow re-synthesizes and optimizes the design for each period, changing the netlist. In run 7, the critical path ended at `p[6]` and passed through several adder-type cells. A `maj3_1` cell took 0.89 ns in the slow corner. Clock uncertainty was 0.25 ns and setup time was 0.293 ns. At 7.0 ns, the worst setup slack was +0.335 ns.

10.5 Final results (run 8, 7.0 ns)

Table 6: Part D results.

Metric	Value
Clock period	7.0 ns
Number of runs	8
Die area	22,500 μm^2 (150×150)
Core area	17,759.5 μm^2
Cells after synthesis	73
Cells after place and route	1,896 (334 excluding decap and fill)
Added cells	245 tap, 1,313 decap, 249 fill, 3 clock buffers, about 19 timing-repair buffers
Wirelength	2,375 μm
Worst setup slack	+0.335 ns
Post-route $F_{\max}$	150.0 MHz
Part C $F_{\max}$	385.2 MHz
DRC errors	0
LVS errors	0

10.6 Cell count: after synthesis and after place and route

The design grew from 73 to 1,896 cells, mainly because physical tap, decap, and fill cells were added. The table lists the final cell groups.

The counts add up to 1,896 cells: 245 tap cells and 89 other standard cells make 334, followed by 1,562 decap and fill cells. LibreLane performs its own synthesis, so its logic differs slightly from the 73-cell Part B netlist. The synthesized cell area of 805.773 μm^2 is about 4.5% of the 17,759.5 μm^2 core area, which explains the large amount of filler.

10.7 The layout

Figures 4 and 5 show the final GDSII layout, rendered with the SKY130 layer colours.

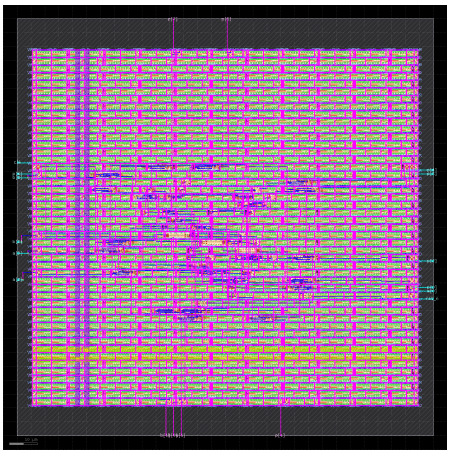


Figure 4: The complete 150 μm × 150 μm die. The cell rows fill the core, while the power grid appears as vertical and horizontal stripes. The input and output pins (`clk`, `rst_n`, `a`, `b`, and `p`) are located along the die edges, with signal routing in the middle. The scale bar at the bottom left represents 10 μm.

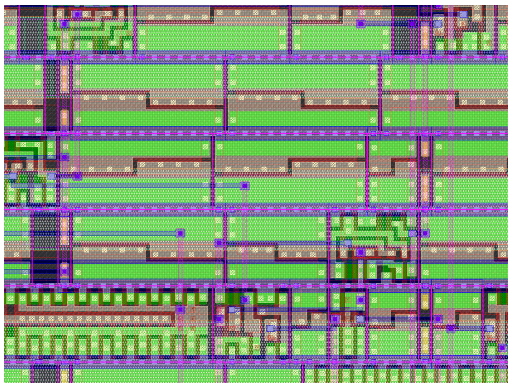


Figure 5: Zoomed view of an approximately 18 μm × 12 μm area near the die center. Standard-cell rows run horizontally, with power rails between them. Horizontal and vertical metal wires cross the cells, while vias connect the metal layers.

10.8 Post-route F_{max} compared with Part C

The post-route maximum frequency is $1000/(7.0 - 0.335) \approx 150.0$ MHz, compared with 385.2 MHz in Part C. The layout is about 2.6 times slower, mainly because signoff includes the slow corner, extracted wire delays, clock uncertainty, and a larger setup time.

Table 7: Part C and Part D timing comparison.

Effect	Part C	Part D signoff
Corner	Typical (25°C, 1.8 V)	Includes slow (100°C, 1.6 V)
Gate delay	`maj3_1`: 0.375 ns	`maj3_1`: 0.89 ns (slow corner)
Wires	Ideal, zero delay	Extracted parasitics; 2,375 μm wirelength
Clock	Ideal, zero skew	Real clock tree with skew
Clock uncertainty	None	0.25 ns
Flip-flop setup	0.135 ns	0.293 ns (slow corner)

At 3.5 ns, the typical-corner slack was +0.568 ns, giving an estimated typical-corner limit of about 340 MHz. This is closer to Part C's result, suggesting the slow signoff corner is the main reason for the lower final frequency. This comparison is approximate because each run synthesizes the design again.

10.9 Observations

- **Routing:** The first routing iteration had 22 DRC violations; the second removed them all.

- **Wire length:** The final route is 2,375 μm, about 5% above the estimated 2,262.63 μm. The longest net was 153.24 μm.
- **Die size:** The design uses only a small part of the fixed 150×150 μm die, so congestion is low but many fill cells are needed.
- **Buffers:** No hold- or setup-repair buffers were reported. The 19 timing-repair buffers were likely added to improve slew and load; this cause was not confirmed.
- **Signoff:** The final layout has zero DRC errors, zero LVS errors, and positive worst setup slack.
- **Main lesson:** The initial $1.2 \times T_{min}$ estimate was not enough. Physical signoff must account for real corners, wires, and the clock tree.

11 Answers to the questions

Question 1

The array needs 16 AND gates and 12 full adders. How many cells did synthesis produce for the logic, and why is the number not simply $16 + 12$*?*

Synthesis produced 73 cells, and 16 of them are flip-flops, so the logic has $73 - 16 = \mathbf{57}$ cells (405.389 μm^2). This is about double the $16 + 12 = 28$ that we might expect. Yosys flattened the design, and ABC then mapped the whole multiplier as one block of logic. The library has full-adder cells, but none was used in our netlist, so every adder was built from smaller cells such as `xor3`, `xnor3`, `maj3` and AND-OR-INVERT gates. Many of the 16 AND gates were also merged into nearby gates (`a21oi`, `nand4`, `o2111a` and so on), so they are no longer separate cells. The adders with a constant carry-in of 0 also became simpler. So the count of 57 comes from the choices of the tool and not from our block diagram.

Question 2

What is the critical path of your design, from which register to which register? How many cells are on it, and how does this compare with the estimate of about $2N$ *full-adder delays?*

The critical path starts at the input-register flip-flop for `b_r[0]` and ends at the output-register flip-flop for `p[6]`. There are **8 logic cells** between them: `and2_0`, `or3_1`, `and4_1`, `o2111a_1`, `o21ai_0`, `xnor3_1`, `maj3_1` and `xor3_1`. The data arrives at 2.460 ns (0.395 ns clock-to-Q plus 2.065 ns of logic), and the setup time is 0.135 ns, so $T_{min} = 2.596$ ns. The count of 8 is the same as the estimate $2N = 8$, but the cells are not eight full adders. Only `xnor3_1`, `maj3_1` and `xor3_1` look like adder cells, and they take 0.827 ns, about 40% of the logic delay. The average cell delay is only about 0.26 ns, which is much less than a ripple full adder, because ABC merged adder and AND logic into complex gates.

Question 3

What percentage of the total cell area is flip-flops? Why is this percentage large for such a small design?

The 16 flip-flops take $16 \times 25.024 = 400.384$ μm^2 out of 805.773 μm^2, which is **49.69%**, almost half. The percentage is large because the arithmetic itself is small. The 57 logic cells are only about 405 μm^2, and most of them are small gates. The registers are a fixed cost, because every input bit and every output bit needs its own flip-flop ($4 + 4 + 8$). One `dfrtp_1` (25.024 μm^2) is about 6.7 times bigger than a `nand2_1` (3.754 μm^2). So in a small design like this, storing the data costs as much area as computing with it. In a bigger arithmetic block the share of the registers would be smaller.

Question 4

Why is the post-route F_{max} *lower than the* F_{max} *from Part C? Which kinds of cells did the flow add, and why is each kind needed?*

The post-route F_{max} is $1000/(7.0 - 0.335) = \mathbf{150.0}$ MHz, and the Part C value is **385.2** MHz. The two analyses are not the same. Part C used the typical corner with ideal wires and an ideal clock. The signoff also checks the slow corner (100°C, 1.6 V), where our `maj3` cell took 0.89 ns instead of 0.375 ns. It also includes real wires, a real clock tree with skew, a 0.25 ns clock uncertainty and a longer setup time (0.293 ns instead of 0.135 ns). The slow corner is the biggest part, because at 3.5 ns the same layout still had +0.568 ns of slack at the typical corner. The flow added these cells: 245 **tap cells** that connect the wells and substrate to the power rails and prevent latch-up; 1,313 **decap cells** and 249 **fill cells** that fill the empty row space and keep the power rails and layer density continuous (the decap cells also keep the supply steady); 3 **clock buffers** so that all 16 flip-flops get the clock at nearly the same time; and about 19 **timing-repair buffers** that fix slew and setup problems on long or heavily loaded nets.

Question 5

What does a DRC check verify, and what does an LVS check verify? Why must both pass before a chip can be manufactured?

A DRC (design rule check) compares the layout shapes with the design rules of the foundry, such as minimum width, minimum spacing and enclosure, so that the shapes can be printed and etched correctly. An LVS (layout versus schematic) check extracts the transistors and connections from the layout and compares them with the netlist, to confirm that the layout is the circuit we designed. The two checks find different mistakes. A layout can pass DRC and still be the wrong circuit, for example with a missing wire or a short between two nets. A layout can also be the right circuit and still break a rule, so it cannot be manufactured well. A chip is safe to send for manufacturing only when it can be built (DRC) and is correct (LVS). In our final run both were clean: 0 DRC errors from the router, Magic and KLayout, and 0 LVS errors of every kind.

Question 6

Using the formulas in Section 2, estimate how the number of full adders and the critical path delay would change if the multiplier were 8 bits wide instead of 4. Show your working.

An N-bit array multiplier has N^2 AND gates and $N(N-1)$ full adders, and its longest path is about $2N$ full-adder delays. For $N = 8$ this gives 64 AND gates (instead of 16) and $8 \times 7 = \mathbf{56}$ full adders (instead of $4 \times 3 = 12$), which is about 4.7 times as many. The flip-flops go from 16 to 32 ($8 + 8$ on the inputs and 16 on the product). The critical path goes from $2 \times 4 = 8$ to $2 \times 8 = \mathbf{16}$ full-adder delays, so the logic delay doubles. Using our Part C logic delay of 2.065 ns, the 8-bit logic needs about $2 \times 2.065 = 4.13$ ns. The register overhead stays the same (0.395 ns clock-to-Q and 0.135 ns setup, 0.53 ns together), so $T_{min} \approx 4.13 + 0.53 = \mathbf{4.66}$ ns, which is about **215** MHz instead of 385 MHz. So the area grows with N^2, but the delay grows only linearly. This is only an estimate, because the synthesis tool would map a bigger design differently.

12 Conclusion

We designed a 4-bit unsigned array multiplier using 16 AND gates, 12 full adders, and registers on the inputs and output. The design passed all 256 input combinations in both RTL and gate-level simulation. An intentionally faulty version failed 95 tests, confirming that the testbench detects errors. Synthesis produced 73 cells with an area of 805.773 μm^2. Typical-corner timing analysis gave a minimum period of 2.596 ns (385.2 MHz). After place and route, the design passed DRC and LVS at 7.0 ns, with a worst setup slack of +0.335 ns and a post-route frequency of about 150.0 MHz.

Table 8: The same design at each stage.

Stage	Cells	Area	Speed	What it adds
RTL (Part A)	16 AND, 12 full adders, 16 flip-flops	n/a	n/a	Function only
Synthesis (Part B)	73	805.773 μm^2	n/a	Cells and area
Typical-corner STA (Part C)	73	805.773 μm^2	385.2 MHz	Cell delays; ideal wires and clock
Place and route (Part D)	1,896 (334 excluding decap and fill)	Core: 17,759.5 μm^2	150.0 MHz	Wires, clock tree, slow corner, filler

What we learned.

1. *Synthesis changes the logic.* The RTL structure became 57 mapped logic cells, showing why it is useful to inspect the synthesized netlist.

2. *Physical timing is different.* The estimated frequency dropped from 385.2 MHz to 150.0 MHz after signoff, mainly because signoff includes the slow corner, real wires, and the clock tree. The initial 3.5 ns period failed, and eight runs were needed to find a passing period.

3. *Physical overhead matters.* Flip-flops use about half of the synthesized area, while decap and fill cells make up most of the final layout cell count.

4. *Automatic checks help.* The same testbench checked RTL and gate-level behaviour, while DRC and LVS checked the final layout.

Limitations. Part C used one typical corner, an ideal clock, and no wire delay. We did not run our own hold analysis. The clock period was tested in 0.5 ns steps, so 7.0 ns is a passing value but may not be the minimum possible. Results between 5.5 and 6.5 ns did not improve consistently. We did not confirm the reason for this behaviour. Part B and Part D cell counts also differ because LibreLane runs synthesis with its own settings.

Possible improvements. A Wallace or Dadda tree with a fast final adder could improve speed. Adding pipeline registers could also improve timing, but would increase latency and area. Allowing the

tool to choose a smaller die could reduce the number of decap and fill cells.

Reproducibility. The workflow and required files are documented in the repository.

References

[1] N. Weste and D. Harris, *CMOS VLSI Design: A Circuits and Systems Perspective*, 4th ed., Pearson. Chapter 11, datapath subsystems (adders and multipliers).

[2] J. Rabaey, A. Chandrakasan and B. Nikolić, *Digital Integrated Circuits: A Design Perspective*, 2nd ed., Pearson. Arithmetic building blocks and timing.

[3] Icarus Verilog: steveicarus.github.io/iverilog

[4] GTKWave: gtkwave.github.io/gtkwave

[5] Yosys: yosyshq.readthedocs.io/projects/yosys

[6] OpenSTA: github.com/parallaxsw/OpenSTA (manual: doc/OpenSTA.pdf)

[7] LibreLane: github.com/librelane/librelane

[8] OpenROAD: openroad.readthedocs.io

[9] KLayout: klayout.de/doc.html

[10] Magic and Netgen: opencircuitdesign.com/magic and opencircuitdesign.com/netgen

[11] SkyWater SKY130 PDK: skywater-pdk.readthedocs.io

[12] IIC-OSIC-TOOLS: github.com/iic-jku/IIC-OSIC-TOOLS

Contributions

Veekshita Sai: Led the work on Part A (RTL Design and Verification) and Part B (Logic Synthesis using Yosys), including design verification and synthesis analysis. Also reviewed Parts C and D and contributed corrections.

Chandini Gayathri: Led the work on Part C (Static Timing Analysis using OpenSTA) and Part D (Physical Design: RTL to GDSII), including timing analysis and physical implementation. Also reviewed Parts A and B and contributed corrections.

Both members collaborated throughout the project, sharing ideas, reviewing results, and refining each other's work. This collaborative approach ensured that both members developed an understanding of the complete RTL-to-GDSII flow.

13 Project repository

The complete source code, scripts, reports, and layout evidence are available at:

`https://github.com/Chandini1607/array_-multiplier`

‘‘‘